Development Standards & Guidelines

Introduction

- Purpose of this document: Provide a unified set of standards for all development teams.
- Scope: Applies to frontend, backend, DevOps, and QA teams.
- Principles: Consistency, maintainability, readability, scalability.

Coding Standards

- Languages & Frameworks: Node.js, NestJS, React, SQL.
- Formatting & Style: Use Prettier/ESLint, 2 spaces indentation, camelCase for variables, PascalCase for classes, snake_case for database tables.
- Best Practices: Avoid hardcoding configs (use .env), write modular and reusable code, document functions with comments/JSDoc.

Version Control (Git)

- Branching Strategy: main, develop, feature branches.
- Commit Messages: Follow Conventional Commits (feat:, fix:, chore:).
- Pull Requests: Require review and approvals, CI checks must pass.

Testing & Quality Assurance

- Unit Testing: Jest for Node/NestJS.
- Integration Testing as needed.
- Test Coverage Minimum: 80%.
- Code Reviews: At least 1–2 reviewers per PR.

Database Standards

- Naming Conventions: snake_case for tables and columns.
- Schema Changes: Always use migrations, never direct edits.
- Indexing Strategy for performance.
- ORM Best Practices: TypeORM usage.

Security Guidelines

- Never commit secrets to Git.
- Passwords must be hashed (scrypt/bcrypt).
- Validate and sanitize all user input.

- Use HTTPS and secure headers (Helmet for Express/NestJS).

DevOps & Deployment

- Maintain separate environments: dev, staging, production.
- CI/CD Pipeline: build → test → deploy.
- Logging & Monitoring: Winston, PM2, Plesk logs.
- Rollback Procedures must be documented.

Frontend Standards

- Component Structure: React/EJS templates.
- Styling: CSS variables and theming (light/dark modes).
- Responsive Design: Mobile-first approach.
- Accessibility: ARIA roles, keyboard navigation.

Documentation

- README requirements for each repository.
- API Documentation: Swagger for NestJS.
- Commenting Practices: Concise and meaningful.
- Onboarding Guide: Clear steps for new developers.

Communication & Workflow

- Daily standups and weekly syncs.
- Task Tracking: Jira, Trello, or GitHub Projects.
- Code of Conduct: Respectful and constructive reviews.
- Collaboration Tools: Slack/Teams etiquette.

```
# .env file example
DB_HOST=localhost
DB_PORT=5432
DB_USER=myuser
DB_PASS=securepassword
```

```
feat(auth): add JWT-based authentication
fix(api): correct null pointer in user service
```

```
// Example Jest Test
import { sum } from './math';

test('adds 1 + 2 to equal 3', () => {
```

```
  expect(sum(1, 2)).toBe(3);
});

# Example TypeORM Migration
import { MigrationInterface, QueryRunner } from "typeorm";

export class CreateUsersTable1693252000000 implements
MigrationInterface {
  public async up(queryRunner: QueryRunner): Promise<void> {
    await queryRunner.query(`CREATE TABLE users (id SERIAL PRIMARY KEY,
name VARCHAR(100))`);
  }

  public async down(queryRunner: QueryRunner): Promise<void> {
    await queryRunner.query(`DROP TABLE users`);
  }
}
```

Architecture Principles (React + NestJS + Clean Architecture)

- Separation of concerns: domain/application/infrastructure/interface layers.
- Feature-first modules: each feature owns API, handlers, repository interfaces, and UI screens.
- CQRS: split read models (queries) from write models (commands).
- Repositories abstract persistence; Prisma is the default implementation.
- All I/O behind interfaces; pure domain logic is framework-agnostic and unit-testable.
- Authentication via JWT (access + refresh) and http-only cookies; Authorization via roles & policies.
- Validation at boundaries: DTOs (class-validator) on input, typed responses (DTOs) on output.

Frontend Standards (React + TypeScript)

Project Structure

```
apps/web/
  src/
    app/              # App shell, routing, auth provider
    features/
      users/
        components/
        pages/
        api/          # API client for this feature
        hooks/
        types.ts
    shared/
      components/
```

```
      hooks/
      lib/            # http client, config
      ui/             # design system wrappers
    assets/
    styles/
  vite.config.ts | next.config.js
  tsconfig.json
  .eslintrc.cjs
```

HTTP Client and API Layer

```
// src/shared/lib/http.ts
export const API_URL = import.meta.env.VITE_API_URL;

export async function http<T>(path: string, init?: RequestInit):
Promise<T> {
  const res = await fetch(`${API_URL}${path}`, {
    credentials: 'include',
    headers: { 'Content-Type': 'application/json', ...(init?.headers ||
{}) },
    ...init,
  });
  if (!res.ok) {
    const err = await res.json().catch(() => ({ message: res.statusText
}));
    throw Object.assign(new Error(err.message || 'Request failed'), {
status: res.status, details: err });
  }
  return res.json() as Promise<T>;
}
```

Feature API Client Example

```
// src/features/users/api/usersApi.ts
import { http } from '@/shared/lib/http';
import type { User, CreateUserDto } from '../types';

export const UsersApi = {
  list: () => http<User[]>('/users'),
  get: (id: string) => http<User>(`/users/${id}`),
  create: (dto: CreateUserDto) => http<User>('/users', { method:
'POST', body: JSON.stringify(dto) }),
};
```

Component + Accessibility Example

```
// src/features/users/components/UserCard.tsx
import type { User } from '../types';
```

```tsx
type Props = { user: User; onOpen?: (id: string) => void };

export function UserCard({ user, onOpen }: Props) {
  return (
    <article role="group" aria-labelledby={`user-${user.id}-name`}
className="rounded-2xl p-4 border">
      <h3 id={`user-${user.id}-name`} className="text-lg font-
semibold">{user.fullName}</h3>
      <p className="text-sm text-muted-foreground">{user.email}</p>
      <button
        type="button"
        className="mt-3 px-3 py-2 rounded-xl border"
        onClick={() => onOpen?.(String(user.id))}
        aria-label={`Open details for ${user.fullName}`}
      >
        View
      </button>
    </article>
  );
}
```

Global Styles & CSS Variables

```css
:root{
  --bg:#ffffff; --text:#0f1724; --primary:#1476f2; --
border:rgba(0,0,0,.12);
}
.dark{ --bg:#0b1220; --text:#e7eefc; --primary:#7aa8ff; }
body{ background:var(--bg); color:var(--text); }
```

Backend Standards (NestJS + Clean Architecture + CQRS + Prisma)

Layered Structure

```
apps/api/
  src/
    modules/
      users/
        application/
          commands/
            create-user.command.ts
            create-user.handler.ts
          queries/
            get-user.query.ts
            get-user.handler.ts
          dto/
```

```
            create-user.dto.ts
            user.response.ts
          ports/
            user.repository.ts
        domain/
          user.entity.ts
          user.errors.ts
        infrastructure/
          persistence/
            prisma-user.repository.ts
        interface/
          rest/
            users.controller.ts
    shared/
      auth/
      config/
      infra/
    prisma/
      schema.prisma
```

Prisma Schema (excerpt)

```
// prisma/schema.prisma
generator client { provider = "prisma-client-js" }
datasource db { provider = "postgresql"; url = env("DATABASE_URL") }

model User {
  id           String   @id @default(cuid())
  email        String   @unique
  passwordHash String
  fullName     String
  role         Role     @default(USER)
  createdAt    DateTime @default(now())
  updatedAt    DateTime @updatedAt
  tokens       RefreshToken[]
}

model RefreshToken {
  id        String   @id @default(cuid())
  userId    String
  token     String   @unique
  expiresAt DateTime
  createdAt DateTime @default(now())
  user      User     @relation(fields: [userId], references: [id],
onDelete: Cascade)
}

enum Role {
  USER
```

```
  ADMIN
}
```

Repository Port (Interface)

```
// users/application/ports/user.repository.ts
import { User } from '../../domain/user.entity';

export interface IUserRepository {
  findById(id: string): Promise<User | null>;
  findByEmail(email: string): Promise<User | null>;
  create(user: User): Promise<User>;
}
export const IUserRepository = Symbol('IUserRepository');
```

Domain Entity

```
// users/domain/user.entity.ts
export class User {
  constructor(
    public readonly id: string,
    public email: string,
    public passwordHash: string,
    public fullName: string,
    public role: 'USER' | 'ADMIN' = 'USER',
  ) {}
}
```

Prisma Repository Implementation

```
// users/infrastructure/persistence/prisma-user.repository.ts
import { Injectable } from '@nestjs/common';
import { PrismaService } from '../../../shared/infra/prisma.service';
import { IUserRepository } from
'../../application/ports/user.repository';
import { User } from '../../domain/user.entity';

@Injectable()
export class PrismaUserRepository implements IUserRepository {
  constructor(private prisma: PrismaService) {}
  async findById(id: string): Promise<User | null> {
    const r = await this.prisma.user.findUnique({ where: { id } });
    return r ? new User(r.id, r.email, r.passwordHash, r.fullName,
r.role) : null;
  }
  async findByEmail(email: string): Promise<User | null> {
    const r = await this.prisma.user.findUnique({ where: { email } });
    return r ? new User(r.id, r.email, r.passwordHash, r.fullName,
```

```
r.role) : null;
  }
  async create(u: User): Promise<User> {
    const r = await this.prisma.user.create({ data: {
      id: u.id, email: u.email, passwordHash: u.passwordHash, fullName:
u.fullName, role: u.role
    }});
    return new User(r.id, r.email, r.passwordHash, r.fullName, r.role);
  }
}
```

CQRS Command + Handler

```
// users/application/commands/create-user.command.ts
export class CreateUserCommand {
  constructor(
    public readonly email: string,
    public readonly password: string,
    public readonly fullName: string,
  ) {}
}

// users/application/commands/create-user.handler.ts
import { CommandHandler, ICommandHandler } from '@nestjs/cqrs';
import { CreateUserCommand } from './create-user.command';
import { IUserRepository } from '../ports/user.repository';
import * as bcrypt from 'bcryptjs';
import { randomUUID } from 'crypto';
import { User } from '../../domain/user.entity';

@CommandHandler(CreateUserCommand)
export class CreateUserHandler implements
ICommandHandler<CreateUserCommand> {
  constructor(private readonly users: IUserRepository) {}
  async execute(cmd: CreateUserCommand) {
    const existing = await this.users.findByEmail(cmd.email);
    if (existing) throw new Error('Email already in use');
    const hash = await bcrypt.hash(cmd.password, 12);
    const user = new User(randomUUID(), cmd.email, hash, cmd.fullName,
'USER');
    return this.users.create(user);
  }
}
```

DTO + Validation

```
// users/application/dto/create-user.dto.ts
import { IsEmail, MinLength } from 'class-validator';
export class CreateUserDto {
```

```
  @IsEmail() email!: string;
  @MinLength(8) password!: string;
  @MinLength(2) fullName!: string;
}
```

Controller using CommandBus/QueryBus

```
// users/interface/rest/users.controller.ts
import { Body, Controller, Get, Param, Post, UseGuards } from
'@nestjs/common';
import { CommandBus, QueryBus } from '@nestjs/cqrs';
import { CreateUserCommand } from '../../application/commands/create-
user.command';
import { CreateUserDto } from '../../application/dto/create-user.dto';
import { JwtAuthGuard } from '../../../shared/auth/jwt.guard';
import { Roles } from '../../../shared/auth/roles.decorator';
import { RolesGuard } from '../../../shared/auth/roles.guard';

@Controller('users')
@UseGuards(JwtAuthGuard, RolesGuard)
export class UsersController {
  constructor(private cmd: CommandBus, private qry: QueryBus) {}
  @Post()
  @Roles('ADMIN')
  create(@Body() dto: CreateUserDto) {
    return this.cmd.execute(new CreateUserCommand(dto.email,
dto.password, dto.fullName));
  }
  @Get(':id')
  get(@Param('id') id: string) {
    // return this.qry.execute(new GetUserQuery(id));
    return { id }; // placeholder
  }
}
```

Authentication & Authorization

- Use Passport JWT strategy for access token; refresh token stored in http-only cookie.
- Protect routes with JwtAuthGuard; apply RoleGuard or CASL policies for fine-grained authorization.

```
// shared/auth/roles.decorator.ts
import { SetMetadata } from '@nestjs/common';
export const ROLES_KEY = 'roles';
export const Roles = (...roles: ('USER'|'ADMIN')[]) =>
SetMetadata(ROLES_KEY, roles);
```

```
// shared/auth/roles.guard.ts
import { CanActivate, ExecutionContext, Injectable } from
'@nestjs/common';
import { Reflector } from '@nestjs/core';
import { ROLES_KEY } from './roles.decorator';

@Injectable()
export class RolesGuard implements CanActivate {
  constructor(private reflector: Reflector) {}
  canActivate(ctx: ExecutionContext): boolean {
    const roles = this.reflector.getAllAndOverride<(
'USER'|'ADMIN')[]>(ROLES_KEY, [
      ctx.getHandler(), ctx.getClass()
    ]);
    if (!roles?.length) return true;
    const req = ctx.switchToHttp().getRequest();
    const user = req.user as { role?: 'USER'|'ADMIN' };
    return !!user && roles.includes(user.role ?? 'USER');
  }
}
```

Error Response Contract

```
// Standard error shape
{
  "error": {
    "code": "VALIDATION_ERROR" | "UNAUTHORIZED" | "FORBIDDEN" |
"NOT_FOUND" | "CONFLICT",
    "message": "Human readable",
    "details": { /* optional field errors */ }
  }
}
```

Migrations & CI

- Prisma: use `prisma migrate dev` locally and `prisma migrate deploy` in CI/CD.
- Pre-deploy checks: build, lint, test, database connectivity smoke test.
- Swagger (OpenAPI) generated at `/docs` in non-production; pinned versions in production.

PostgreSQL Standards & Practices

Environment & Connection

- Use a single DATABASE_URL per environment; never hardcode credentials.
- Enforce SSL in production; allow self-signed only if your provider requires it and you trust the cert.

- Keep max open connections low (Prisma + Node) and prefer a pooler (pgBouncer) in transaction mode.
- Store all timestamps in UTC using timestamptz.

```
# .env (example)
DATABASE_URL="postgresql://app_user:strong_password@db.host:5432/app_db
?schema=public&sslmode=require"

# Local dev (no SSL)
DATABASE_URL="postgresql://app_user:dev_password@localhost:5432/app_db?
schema=public"
```

Prisma + Postgres Conventions

- Use UUID primary keys generated in the database.
- Snake_case in the database; map to camelCase in Prisma models with @map/@@map.
- Always include createdAt/updatedAt (timestamptz).
- Use explicit relations with referential actions (onDelete/onUpdate).

```
// prisma/schema.prisma (excerpt with snake_case mapping)
generator client { provider = "prisma-client-js" }
datasource db { provider = "postgresql"; url = env("DATABASE_URL") }

model User {
  id           String   @id @default(dbgenerated("gen_random_uuid()"))
@db.Uuid  @map("id")
  email        String   @unique
  passwordHash String   @map("password_hash")
  fullName     String   @map("full_name")
  role         Role     @default(USER)
  createdAt    DateTime @default(now())     @db.Timestamptz(6)
@map("created_at")
  updatedAt    DateTime @updatedAt          @db.Timestamptz(6)
@map("updated_at")
  tokens       RefreshToken[]

  @@map("users")
  @@index([email])
}

model RefreshToken {
  id        String   @id @default(dbgenerated("gen_random_uuid()"))
@db.Uuid
  userId    String   @db.Uuid   @map("user_id")
  token     String   @unique
  expiresAt DateTime @db.Timestamptz(6)     @map("expires_at")
  createdAt DateTime @default(now()) @db.Timestamptz(6)
@map("created_at")
```

```
  user      User     @relation(fields: [userId], references: [id],
onDelete: Cascade)

  @@map("refresh_tokens")
  @@index([userId])
}

enum Role {
  USER
  ADMIN
}
```

Database Setup & Extensions

- Enable required extensions: pgcrypto (for gen_random_uuid), pg_trgm for fuzzy search when needed.
- Prefer generated columns or triggers for complex derived data when it lives in DB.

```
-- One-time setup (run as superuser/admin)
CREATE EXTENSION IF NOT EXISTS pgcrypto; -- gen_random_uuid()
CREATE EXTENSION IF NOT EXISTS pg_trgm;  -- trigram indexes for
fuzzy/text search
```

Indexing & Performance

- Index foreign keys and frequent filter columns.
- Use partial indexes for sparse boolean flags; GIN indexes for JSONB or text search.
- Validate plans with EXPLAIN ANALYZE on slow queries.
- Avoid N+1 by joining/including relations in queries where appropriate.

```
-- Examples
-- FK index (often automatic via Prisma migrations but verify)
CREATE INDEX IF NOT EXISTS idx_refresh_tokens_user_id ON
refresh_tokens(user_id);

-- Trigram index for ILIKE '%text%'
CREATE INDEX IF NOT EXISTS idx_users_email_trgm ON users USING GIN
(email gin_trgm_ops);

-- JSONB GIN index
-- CREATE INDEX idx_table_data_gin ON my_table USING GIN (data
jsonb_path_ops);
```

Migrations Strategy

- All schema changes via Prisma Migrate: `prisma migrate dev` (local), `prisma migrate deploy` (CI/CD).

- Never edit production schema manually.
- Review SQL in each migration; keep them in VCS.

```
# Local dev
npx prisma migrate dev --name add_users

# Production/CI
npx prisma migrate deploy

# Generate client after changes
npx prisma generate
```

Transactions & Concurrency

- Wrap multi-step operations in a transaction (`prisma.$transaction`).
- Choose appropriate isolation level when needed (REPEATABLE READ for read consistency, SERIALIZABLE for strict).

```
// NestJS service (simplified)
await this.prisma.$transaction(async (tx) => {
  const user = await tx.user.create({ data: {/*...*/} });
  await tx.refreshToken.create({ data: {/*...*/} });
  return user;
});
```

Security & Access Control

- Create least-privilege roles: app_user for application, read_only for analytics, separate admin role.
- Restrict schema permissions; never use superuser from the app.
- Rotate credentials; store secrets only in env/secret manager.

```
-- Roles and grants (example)
CREATE ROLE app_user LOGIN PASSWORD '***';
CREATE ROLE read_only;

GRANT USAGE ON SCHEMA public TO app_user, read_only;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO
app_user;
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT, INSERT, UPDATE,
DELETE ON TABLES TO app_user;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO read_only;
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT ON TABLES TO
read_only;
```

Backups & Restore

- Automate daily logical backups; keep at least 7–14 days of retention.
- Test restores regularly in a staging environment.

```
# Backup
pg_dump --format=custom --file=backup_$(date +%F).dump --
dbname="$DATABASE_URL"

# Restore
pg_restore --clean --if-exists --dbname="$DATABASE_URL" backup_2025-09-
03.dump
```

Time Zones & Locale

- Store all timestamps as timestamptz (UTC).
- Convert to user timezone at the UI/API edge as needed.